FACULDADE SENAC DF · UNIDADE TECNOLOGIA · BRASÍLIA

StockFlow

Relatório de evolução do projeto: da concepção ao estado atual

Plataforma SaaS multiempresa de gestão inteligente de estoque para pequenas e médias empresas

Curso	Análise e Desenvolvimento de Sistemas (ADS-2A) · 2º semestre
Disciplina	Laboratório de Inovação II
Equipe	Bruno Sampaio · Daniel Oliveira · Elinaldo · Felipe Marra · Josafa Felinto · Raphael Feitoza
Repositório	github.com/BCZero/stock-flow-gestao-inteligente
Posição	Branch `main`, commit `0fee0cd` · Fases 1–5 entregues, Fase 6 parcial
Data	09 de novembro de 2026

1. Resumo

O **StockFlow** é um sistema web de gestão de estoque voltado a pequenos varejistas. Cada empresa-cliente opera em um ambiente logicamente isolado (arquitetura *multi-tenant*) e conta com um motor transacional que impede saldo negativo mesmo com vendas simultâneas, um histórico imutável de movimentações, uma trilha de auditoria e alertas automáticos de estoque crítico, produto parado, atraso de fornecedor e variação de vendas.

Este relatório descreve **como o projeto evoluiu** desde o primeiro commit (08/10/2026) até o estado atual, o que foi planejado e o que foi efetivamente entregue, as principais decisões de engenharia, **as verificações executadas** e as limitações que ainda existem. Todas as afirmações foram conferidas contra o código-fonte, o histórico do Git e uma execução da suíte de testes em 09/11/2026.

5 + ½ fases entregues (Fase 6 parcial)	**17** tabelas, em 3 migrações	**34** rotas da API REST	**10 / 10** testes automatizados aprovados
10 telas no frontend	**≈ 4,9 mil** linhas de código (back 3.093 · front 1.779)	**4** perfis de acesso (RBAC)	**5** tipos de alerta automático

2. Contexto e problema

O projeto nasceu das oficinas de **Laboratório de Inovação II**, cujo desafio era conceber, modelar e implementar uma solução de software completa, integrando modelagem relacional com integridade rigorosa, controle de acesso por perfis, tecnologias web modernas e entrega incremental em seis fases. O ponto de partida foi um *Projeto Lógico e Físico de Banco de Dados*, um Plano de Negócios e um Project Model Canvas.

O problema atacado é recorrente no pequeno varejo: controles em planilhas, sem rastreabilidade (“quem alterou este saldo?”), vendas que superam o estoque físico, capital imobilizado em produtos parados e reação tardia a mudanças de demanda e a atrasos de fornecedores.

3. Linha do tempo da evolução

08/10/2026 `2884af6 · 6afe185 · 349b831 · c52b5d7`

Fundação (Fases 1 a 3). Monorepo (`backend/` e `frontend/`), PostgreSQL 16 em Docker, FastAPI com Alembic (migração inicial de 9 tabelas), cadastro atômico de empresa + administrador, login JWT com 2FA (TOTP), CRUDs de categorias, produtos, locais e pessoas e interface React com rotas protegidas. Um commit posterior tirou do versionamento o ambiente virtual enviado por engano e criou o `.gitignore` .

05/11/2026 `e970e4f`

Suporte operacional. Utilitário de linha de comando `gerenciar_usuario.py` (listar usuários, redefinir senha, desativar 2FA de emergência), para recuperar contas durante o desenvolvimento e as apresentações.

06/11/2026 `14c2957 · 38 arquivos · +3.458 linhas`

O coração do sistema (Fases 4 e 5). Motor transacional de estoque (compras com recebimento parcial, vendas com baixa atômica, ajustes com motivo), ledger e auditoria imutáveis, painel analítico, monitor de alertas, notificações, resumo opcional por LLM, carga de demonstração e a primeira suíte de testes — o maior salto técnico do projeto.

09/11/2026 `2297919 · 0fee0cd`

Estabilização e documentação. Login por nome ou e-mail, falha de auditoria deixando de derrubar o login, diagnóstico no CLI, primeira documentação completa — e, nesta data, a verificação independente descrita na seção 7.

3.1. Planejado × entregue

Fase (plano de trabalho)	Entrega verificada no código	Situação
1. Ambiente e banco	Docker Compose com PostgreSQL 16, SQLAlchemy, Alembic, React + Vite + Axios com interceptor de token.	Concluída
2. Autenticação, perfis e 2FA	Registro de empresa + admin, login JWT, 2FA TOTP com QR Code, quatro perfis (ADMIN, GESTOR, OPERADOR, ANALISTA).	Concluída
3. Cadastros essenciais	Categorias, produtos (SKU único por empresa), locais e pessoas (clientes/fornecedores); isolamento por empresa. *Validação de CPF/CNPJ ficou pendente.*	Concluída
4. Estoque e transações	Compras (rascunho → confirmada → parcial → recebida), vendas, ajustes, saldos por local, histórico, bloqueio de saldo negativo e idempotência.	Concluída
5. Inteligência, alertas e auditoria	Painel de 30 dias, 5 tipos de alerta, notificações, fluxo de tratamento, regras por empresa, trilha de auditoria, resumo por LLM.	Concluída
6. Testes e validação	Concorrência, isolamento e *fallback* de LLM cobertos por testes; dados simulados por `popular_demo.py` . *Faltam teste de volume, revisão de responsividade e testes do frontend.*	Parcial

4. Arquitetura

O sistema segue uma arquitetura em três camadas: uma aplicação de página única (SPA) em React consome uma API REST em FastAPI, que persiste em PostgreSQL. Um monitor assíncrono, executado dentro do próprio processo da API, gera os alertas periodicamente; um provedor de LLM é opcional.

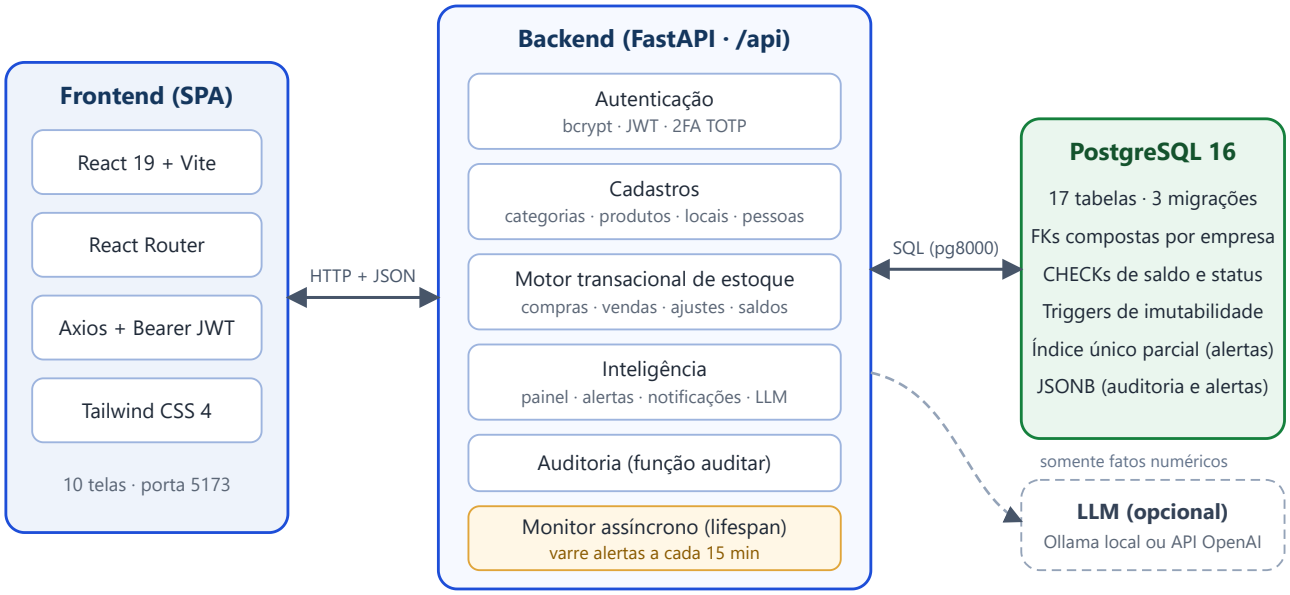


Figura 1 — Visão geral da arquitetura do StockFlow.

4.1. Tecnologias adotadas

Camada	Tecnologia	Por que foi escolhida
API	Python 3.11+ · FastAPI · Pydantic 2	Validação tipada, documentação OpenAPI automática e *lifespan* para o monitor.
Dados	PostgreSQL 16 · SQLAlchemy 2 · Alembic	ACID, `CHECK`, *triggers*, índices parciais, JSONB e esquema versionado.
Driver	`pg8000`	Python puro: contornou bloqueios de *Device Guard* do Windows nas máquinas acadêmicas.
Segurança	bcrypt · python-jose · pyotp	Hash com *salt*, JWT assinado e 2FA TOTP (RFC 6238).
Frontend	React 19 · Vite 8 · Tailwind 4 · React Router 7 · Axios	Componentização, *build* rápido e interface responsiva.
Infra local	Docker Compose	Mesma versão do banco para toda a equipe, com um único comando.

5. Decisões de engenharia em destaque

Isolamento multiempresa no banco

Toda tabela de domínio tem `id_empresa` e as relações usam chaves estrangeiras compostas `(id_empresa, id)`. O PostgreSQL recusa (SQLSTATE 23503) vincular registros de empresas diferentes **mesmo que o código da aplicação falhe**. O token JWT carrega só o id do usuário; a empresa é lida do banco.

Saldo nunca negativo

Duas barreiras: a aplicação bloqueia as linhas de saldo (`SELECT … FOR UPDATE`) e recusa saldo posterior < 0; o banco tem `CHECK (quantidade >= 0)`. Os bloqueios seguem ordem determinística (produto, local) para evitar *deadlock*.

Ledger e auditoria imutáveis

`movimentacao_estoque` guarda saldo anterior e posterior de cada operação, com `CHECK` que valida a aritmética. Um *trigger* bloqueia `UPDATE` e `DELETE` nesta tabela e em `evento_auditoria`. Produtos com histórico são desativados, nunca apagados.

Idempotência

Ajustes e recebimentos exigem uma `chave` UUID: reenviar a mesma requisição não duplica o estoque; reutilizar a chave com dados diferentes retorna 409. Confirmar uma venda já confirmada não baixa o estoque de novo.

Alertas sem duplicidade

Índice único parcial por condição ativa + bloqueio por empresa: dois processamentos simultâneos geram **um** alerta. Quando a condição normaliza o alerta se encerra sozinho; se o problema reaparecer, nasce um novo.

IA com degradação graciosa e privacidade

O LLM é opcional e recebe apenas referências temporárias e números — nunca nomes, e-mails ou IDs. Sem configuração, falha de rede ou resposta inválida, o painel usa as orientações das regras, sem erro para o usuário.

5.1. Ciclo de vida dos documentos

Compra	`RASCUNHO` → confirmar → `CONFIRMADA` → receber (parcial) → `PARCIAL` → receber (restante) → `RECEBIDA`. Somente o recebimento altera o saldo.
Venda	`RASCUNHO` → confirmar → `CONFIRMADA`. A baixa de todos os itens ocorre numa única transação; se um item faltar, nada é baixado e a venda segue em rascunho.
Alerta	`NOVO` → `LIDO` → `EM_TRATAMENTO` → `RESOLVIDO` | `DESCARTADO` (encerrar exige justificativa).

5.2. Alertas automáticos

Alerta	Prioridade	Condição (limites configuráveis por empresa)
`ESTOQUE_CRITICO`	1 · Crítica	Saldo ≤ estoque mínimo do produto.
`PRODUTO_PARADO`	2 · Atenção	Há saldo e a última venda foi há ≥ N dias (padrão 60).
`ATRASO_FORNECEDOR`	2 · Atenção	Compra confirmada/parcial com itens pendentes e data prevista vencida além da tolerância.
`QUEDA_VENDAS`	3 · Observação	Vendas caíram ≥ X % entre duas janelas de 30 dias (padrão 20 %).
`AUMENTO_VENDAS`	4 · Oportunidade	Vendas subiram ≥ X % entre as duas janelas.

6. Desvios em relação ao plano original

Registrar o que mudou durante a construção é parte da evolução do projeto. As mudanças abaixo foram decisões de engenharia, não omissões.

Previsto	Realizado	Motivo
Script SQL do Projeto Físico	Modelos SQLAlchemy + migrações Alembic	Esquema versionado e reproduzível em qualquer máquina.
`psycopg`	`pg8000`	Bloqueio de bibliotecas compiladas pelo Windows das máquinas acadêmicas.
Celery ou APScheduler	`asyncio` no *lifespan* da API	Sem dependência extra; o bloqueio por empresa no PostgreSQL já serializa o processamento.
Middleware de auditoria	Função `auditar()` explícita + tabela imutável	Registra o contexto de negócio (documento, antes/depois) e a garantia vem do banco.
Validação de CPF/CNPJ	Não implementada	Pendência registrada na seção 8.
Transferência entre locais	Não implementada	Decisão de escopo ainda em aberto.

7. Verificação e resultados

Em **09/11/2026** a suíte foi executada na máquina de desenvolvimento, contra um PostgreSQL 16 descartável em Docker, com as três migrações aplicadas do zero. Esta é uma execução pontual: **não há integração contínua configurada**.

Verificação	Resultado
`python -m unittest discover -s tests -v`	10 de 10 aprovados em ≈ 6,6 s (6 de integração + 4 do resumo por LLM)
`alembic upgrade head` do zero + `alembic check`	OK três migrações aplicadas; "No new upgrade operations detected"
`npm run build` (frontend)	OK 1.976 módulos; JS de 383 kB (120 kB comprimido)

7.1. O que os testes comprovam

Cenário	Evidência
Concorrência test_vendas_concorrentes…	Duas vendas de 4 unidades disputam um saldo de 5, liberadas ao mesmo instante por uma barreira de *threads*: o resultado é **sempre um 200 e um 409**, com saldo final 1.
Atomicidade test_reverte_venda_inteira…	Venda com um item sem saldo sofre *rollback* completo e continua em rascunho; `DELETE` direto no ledger é bloqueado pelo *trigger*.
Idempotência test_compra_parcial…	Reenviar o mesmo recebimento não duplica estoque; recebimento acima do pendente → 409; confirmar a venda duas vezes baixa uma só.
Isolamento multiempresa test_isolamento_perfil_e_sku	A empresa B não vê nem confirma documentos da A (404); vincular categoria de outra empresa falha na API **e no banco** (SQLSTATE 23503).
Perfis	O perfil ANALISTA consulta, mas não movimenta estoque, não lê a auditoria e não gera resumo (403).
Alertas test_alerta_deduplica…	Processamentos simultâneos geram um único alerta; transições inválidas → 409; encerrar sem justificativa → 422; reincidência cria alerta novo.
LLM ResumoAlertasTest	O provedor recebe só fatos numéricos; falha ou resposta inválida → *fallback* por regras; Ollama remoto é recusado.

7.2. Dados de demonstração

O script `popular_demo.py` cria duas lojas fictícias com 90 dias de operações e cenários determinísticos (estoque crítico, produto parado há 75 dias, queda de 30 % nas vendas, produto sem base anterior, fornecedor em atraso e o mesmo SKU em empresas distintas). A carga é idempotente e serve para apresentações e para exercitar os alertas.

8. Estado atual e limitações conhecidas

> **Estado:** o sistema está **operacional e demonstrável**: login com 2FA, painel, cadastros, estoque, compras, vendas, alertas, notificações, auditoria e resumo por LLM funcionam de ponta a ponta. A Fase 6 está **parcial**.

Na revisão do código foram identificadas as limitações abaixo. Nenhuma impede a demonstração acadêmica; todas devem ser tratadas antes de qualquer uso em produção.

#	Limitação	Correção sugerida
1	`SECRET_KEY` tem valor padrão fixo no código.	Exigir a chave por ambiente e falhar ao iniciar se ausente.
2	CORS aberto (`allow_origins=["*"]`) com credenciais.	Restringir à origem do frontend.
3	Login por **nome** usa `.first()` , mas o nome de usuário não é único.	Tornar o nome único ou aceitar só e-mail.
4	Cadastros (categorias, produtos, locais, pessoas) não verificam o perfil: um ANALISTA pode criar e excluir.	Reutilizar as dependências de perfil do módulo de estoque.
5	O frontend não oculta ações por perfil; só o backend bloqueia (403).	Esconder ou desabilitar ações conforme o perfil.
6	CPF/CNPJ não são validados.	Validar dígitos verificadores no esquema Pydantic.
7	Token em `localStorage` , sem *refresh token*.	Cookie `HttpOnly` + *refresh*.
8	Sem teste de volume, sem testes do frontend, sem CI.	Concluir a Fase 6 e criar pipeline com Postgres de serviço.

8.1. Próximos passos

- **Concluir a Fase 6:** teste de volume com a carga de demonstração ampliada e medição de tempo do painel, dos alertas e das listagens; revisão de responsividade em celular (uso no balcão).
- **Endurecer a segurança e os perfis** (limitações 1 a 6) e cobrir autenticação e cadastros com testes automatizados.
- **Evolução de produto** prevista no Plano de Negócios: previsão de demanda e sugestão de reposição — hoje o sistema *alerta*, mas ainda não *prevê* nem *sugere quantidades*.

9. Como executar

Requisitos: Git, Docker Desktop, Python 3.11+ e Node.js 18+. Em três terminais, a partir da raiz do repositório:

Etapa	Comando
Banco	`docker compose up -d`
Backend	`cd backend` · `python -m venv venv` · `venv\Scripts\python.exe -m pip install -r requirements.txt` · `venv\Scripts\python.exe -m alembic upgrade head` · `venv\Scripts\python.exe -m uvicorn app.main:app --reload` → API em `127.0.0.1:8000` (Swagger em `/docs`)
Frontend	`cd frontend` · `npm install` · `npm run dev` → interface em `localhost:5173`
Primeira conta	`venv\Scripts\python.exe gerenciar_usuario.py` → opção 5 (criar usuário e empresa)
Demonstração	`popular_demo.py` com `STOCKFLOW_DEMO_DATABASE_URL` apontando para o banco `stockflow_demo` (passo a passo completo no `DOCUMENTACAO.md`)

Documentação do repositório

- `DOCUMENTACAO.md` — documentação técnica completa (arquitetura, modelo de dados, API, testes, limitações).
- `README.md` — visão rápida e instruções por fase.
- `plano_de_trabalho_stockflow.md` — cronograma original das seis fases.
- `subsidio/` — Plano de Negócios e Project Model Canvas.

Fontes deste relatório: código-fonte da branch `main` (commit 0fee0cd), histórico do Git, e execução da suíte de testes, das migrações e do *build* do frontend em 09/11/2026.